基于 MongoDB 的问卷系统

第一、二阶段项目完成报告

课程	数据库课程大作业
技术栈	FastAPI + MongoDB + 原生前端
最终测试	268 项通过，代码覆盖率 96%
完成日期	2026-10-28

目录

目录 2

1. 项目目标 . . . 4
2. 需求与界面设计 . . . 4
   2.1 方法：先界面，后数据库 . . . 4
   2.2 关键需求决策 . . . 6
   2.3 问卷状态 . . . 6
3. 系统设计 . . . 6
4. MongoDB 设计（第一阶段基线） . . . 7
   4.1 集合 . . . 7
   4.2 为什么适合 MongoDB，而不用关系型结构 . . . 8
   4.3 索引 . . . 8
   4.4 并发控制 . . . 8
   4.5 冗余的取舍 . . . 9
   4.6 断点续填 . . . 9
5. 跳转设计 . . . 9
6. 校验设计 . . . 10
7. AI 使用过程 . . . 10
   7.1 工具与工作方式 . . . 10
   7.2 AI 帮了什么 . . . 11
   7.3 AI 做错了什么 . . . 12
   7.4 我改了什么、决定了什么 . . . 13
   7.5 体会 . . . 13
8. 测试结果 . . . 13
9. 遇到的问题及解决方法 . . . 14
10. 【第二阶段新增】需求变化与系统演进 . . . 15
    10.1 第一阶段设计和新需求造成的问题 . . . 15
    10.2 修改理由和核心方案 . . . 15
    10.3 数据结构、索引与迁移 . . . 15
    10.4 程序与页面调整 . . . 15
    10.5 兼容性问题和处理 . . . 16
    10.6 如何保证旧功能仍可用 . . . 16
    10.7 第二阶段实施过程 . . . 16

10.8 已知限制 16
10.9 最终提交材料对应关系 17
附录：代表性界面截图 18

【第二阶段修订】本报告保留第一阶段的设计与取舍，并在第 10 节完整说明需求变化、数据迁移、程序修改、兼容边界和旧功能回归。所有测试数字以实际保存的输出为准。

1. 项目目标

开发一个可迭代的在线问卷系统：第一阶段完成注册、问卷编辑、条件跳转、填写与单卷统计；第二阶段在原系统上增加题目身份、不可变版本、共享、多个命名题库、引用查询和同版跨卷统计。按作业要求，重点放在：

- MongoDB 数据库设计：集合划分、文档结构、索引、并发控制，以及设计理由；
- 后端业务逻辑：三种题型的约束校验、可在问卷中配置的跳转逻辑；
- 为第二阶段的需求变更预留扩展空间（第二阶段不允许推倒重做）；
- 全程使用 AI 编程工具，并如实记录使用过程。

最终成果：

项目	数量
后端代码（`backend/app`）	2676 行 Python
前端代码（`frontend`）	1833 行 HTML / JS / CSS，11 个页面
集合校验规则（`schema`）	7 个集合，354 行 JSON
接口	50 个 HTTP 方法/路径组合
自动测试	第二阶段最终 268 个全部通过，代码覆盖率 96%
文档	第一阶段文档全部保留，新增第二阶段设计、迁移记录和逐文件变更清单
演进依据	Git 基线提交 + 实施前工作区快照 + AI 日志 + 六步核验记录

2. 需求与界面设计

2.1 方法：先界面，后数据库

MongoDB 的建模原则是"按应用如何读写数据来组织数据"，所以设计顺序是：需求 → 页面与操作 → 访问模式 → 集合、文档结构、索引。项目刚初始化完，AI 的计划仍是直接设计三个集合。我当即停掉这一步，指出还没有界面分析，并给出自己画的页面流程图，以及题目内嵌、答卷独立、分享设置收成子文档、发布后不真删等数据库要求。之后又就流程和库结构争论了几轮，有的地方我改了自己的流程图，有的地方我否掉了 AI 的方案（见第 7 节，日志 #3 至 #7）。

页面流程由我绘制。讨论中我自己改过的地方包括：填写默认要求登录但保留开关、登录后回到链接入口而不是直达填写页、增加"你已填写过"、草稿外链改为 404。完整说明见需求分析与界面设计：

流程图源代码

```
flowchart LR
    subgraph creator ["■■■■■■■■■■■"]
        Login["■■ / ■■"]
        MyList["■■■■■■■"]
        Detail["■■■■■■"]
        Editor["■■■■■■■ / ■■■■■"]
        Preview["■■■■"]
        Share["■■ / ■■■■■"]
        Stats["■■■■"]
        Export["■■■■■"]
    end

    subgraph personal ["■■■■■■■■■■"]
        Joined["■■■■■■■"]
        MyAnswer["■■■■■■■■■"]
    end

    subgraph filler ["■■■■"]
        Link["■■ /survey/slug"]
        Load{"■■■■■"}
        NotFound["404 / ■■■■■"]
        Closed["■■■■■■"]
        Gate{"■■■■■?"}
        Submitted["■■■■■■"]
        Fill["■■■■"]
        Done["■■■■■■"]
        Thanks["■■■■■■■■■■"]
    end

    Login -->|■■| MyList
    Login -->|"redirect=/survey/slug"| Link

    MyList --> Detail
    MyList -->|■■| Editor
    Detail --> Editor
    Detail --> Stats
    Detail --> Share
    Editor -->|■■ / ■■| Detail
    Editor -->|■■| Preview
    Preview -->|■■■■■| Editor
    Editor -->|■■| Share
    Share --> Link
    Stats --> Export
    Stats -->|■■| Detail

    Link --> Load
    Load -->|"■■■ / ■■■ / ■■"| NotFound
    Load -->|"■■■ / ■■■ / ■■■■■"| Closed
    Load -->|■■■■| Gate
    Gate -->|"■■■■■■■"| Login
    Gate -->|"■■■■■■■"| Submitted
    Submitted -->|"■■■■■ ■ ■■■"| Fill
    Fill -->|■■■■■■| Fill
    Fill --> Done
    Done -->|"■■■■■■"| Thanks

    MyList -->|■■| Joined
    Joined -->|"■■■■■■■■■"| Link
    Joined -->|"■■■■■■■"| MyAnswer
    Fill -.->|"■■■■■■■■■■■■"| Joined
```

填写入口按上图从左到右判断：不存在、已删除或草稿返回 404；已关闭、已过截止时间或答卷数达到上限显示"问卷已结束"；然后才检查登录和"是否已填写过"。每个页面的显示内容和操作被整理成 23 个访问模式（P1 至 P23）。数据库设计中的每个集合、每个索引都对应到具体的访问模式。

2.2 关键需求决策

问题	决定	理由
填写是否需要登录	默认需要；保留开关	需求原文"填写问卷的人需要先登录"；开关为第二阶段预留
"匿名"的含义	登录后填写，但答卷不保存身份	与"必须登录"同时成立
每人一次与免登录	关闭登录要求时必须允许重复填写	不登录无法识别同一个人
发布后能否编辑	可以；删除改为隐藏，题型不可改，结构变更版本加 1	已有答卷引用题目和选项的 ID
跳转范围	只能向后跳	从结构上杜绝死循环
填写方式	一屏一题，下一题由后端计算	跳转逻辑只有一份实现
扩展：断点续填、我参与的问卷	实现（我提出）	改善填写体验，且能体现 TTL、upsert 等 MongoDB 特性

2.3 问卷状态

`status` 只存创作者操作触发的三个状态。"已截止"和"已达上限"由截止时间和答卷数在入口处实时判断，不写入 `status`，因此不依赖定时任务。`deleted` 不是 `status` 的取值，而是 `deletedAt` 有值（软删除），答卷保留。

代码/命令

```
stateDiagram-v2
    [*] --> draft: ■■
    draft --> published: ■■
    published --> closed: ■■
    closed --> published: ■■■■
    draft --> deleted: ■■
    published --> deleted: ■■
    closed --> deleted: ■■
```

3. 系统设计

流程图源代码

```
flowchart LR
    Browser["■■■■HTML + ■■ JS"] -->|"HTTP / JSON"| Routers
    subgraph backend ["FastAPI ■■"]
        Routers["routers■■■■■■■■■"] --> Services["services■■■■■"]
        Services --> Repos["repositories■MongoDB ■■"]
        Services --> Pure["logic_engine / answer_validator / rule_checker■■■■"]
    end
    Repos --> Mongo[("MongoDB■users / surveys / responses / drafts")]
```

- 技术栈：Python 3.11 + FastAPI + PyMongo，MongoDB 9.0.2 单机，bcrypt + JWT，前端为纯 HTML 和原生 JS。
- 分层：MongoDB 查询只出现在 `repositories/` 中，每个函数注明对应的访问模式编号，方便对照设计文档。跳转计算和答案校验是不访问数据库的纯函数，可以单独做单元测试。
- 前端不重复实现规则：跳转和校验只在后端实现，前端每点"下一题"都询问后端。

界面以"答题卡"为视觉语言：印刷红的框线与题号、[A]
括号选项、选中时铅笔涂黑；左侧定位标记显示进度，被跳转跳过的题以斜线标出，让跳转逻辑在填写时可见。

填写页（见截图附录）

详见系统说明。

4. MongoDB 设计（第一阶段基线）

本节保留第一阶段四集合设计和当时的理由，作为第二阶段演进的比较基线。第二阶段新增集合、索引、冻结规则与迁移结果见第 10 节。

详见数据库设计，这里概括要点。

4.1 集合

代码/命令

```
erDiagram
    users ||--o{ surveys : "ownerId"
    surveys ||--o{ responses : "surveyId"
    users ||--o{ responses : "respondentId■■■■■■■■■ null■"
    users ||--o{ drafts : "userId"
    surveys ||--o{ drafts : "surveyId"
    drafts {
        ObjectId userId
        ObjectId surveyId
        object answers "questionId ■■■"
        array path
        date lastVisitedAt "TTL 30 ■"
    }
    surveys {
        ObjectId ownerId
        string slug
        string status
        object settings "■■■■■■■■■■■"
        int responseCount "■■■■■■"
        array questions "■■■■■■■■■■■■■■■■"
    }
    responses {
        ObjectId surveyId
        string submitterKey
        array answers
        array path
    }
```

集合	存什么	设计要点
`users`	账号	用户名唯一索引；只存 bcrypt 哈希
`surveys`	问卷、分享设置、内嵌的全部题目和跳转规则	编辑页、预览、填写都需要整套题目，一次查询拿全、一次更新保存；题型异构，每题只带自己的约束字段
`responses`	每次提交一份答卷	数量无上限、只追加、按问卷聚合，所以单独成集合；答案按原类型存储（数字存数值），记录实际作答路径和问卷版本
`drafts`	填写进度（扩展）	一人一问卷一份、反复覆盖、过期可丢弃，生命周期与答卷不同，所以单独成集合

4.2 为什么适合 MongoDB，而不用关系型结构

访问模式	关系型	本设计
打开问卷（最频繁）	JOIN 题目、选项、约束、规则四五张表，再在程序里拼成树	按 slug 查一个文档，结构与页面一致
保存编辑	多张表增删改，需要事务	更新一个文档，单文档原子
题型异构	大量可空列或 EAV 表	每题只带自己的字段，`$jsonSchema` 限定允许的键
提交答卷	答案拆成多行，多选再拆一层	插入一个文档，多选直接存数组
统计	GROUP BY + JOIN	一条聚合管道
需求变化	改表结构、迁移数据	新字段按默认值兼容，`schemaVersion` 支持渐进迁移

MongoDB 的不足也有对应措施：

- 没有外键：由应用层检查引用；题目和选项只隐藏不删除，问卷只软删除，避免答卷引用悬空。
- 结构灵活容易写入脏数据：四个集合都用 `$jsonSchema` 严格校验。
- 单机部署没有多文档事务：每个业务操作只依赖单文档原子性。

4.3 索引

除默认的 `_id` 外共 9 个索引（users 1 个、surveys 2 个、responses 3 个、drafts 3 个），每个都有访问模式支撑，例如：

- 列表页按 ESR 原则建 `{ownerId, deletedAt, createdAt}`，用 `explain` 验证没有内存排序；
- 统计与导出用 `{surveyId, submittedAt, _id}`；
- "每人一次"用 `{surveyId, submitterKey}` 唯一部分索引（只对 `singleSubmission: true` 的答卷生效）；
- 填写进度用唯一索引和 TTL 索引（30 天自动过期）。

`init_db` 以代码中的索引声明为唯一来源，自动删除旧索引、重建定义变化的索引。

4.4 并发控制

场景	做法
答卷数上限	"检查仍可填写且未满额"与"计数 +1"写在同一条条件更新中；写入答卷失败时归还名额
每人只能填一次	唯一部分索引兜底，应用层的检查只用于给出提示
发布、关闭等状态流转	前置状态和权限写进更新条件，一次原子操作
两个窗口同时编辑	以 `updatedAt` 为乐观锁；`updatedAt` 每次严格递增
填写进度	唯一索引 + upsert；管道更新在一条语句内截断并延长路径

提交时先重放路径、再占名额、后插入答卷。占名额失败或唯一索引冲突都不会留下超额或重复的答卷；插入失败时把名额减回去。本地是单机 MongoDB，没有多文档事务，所以每一步只改一个文档。

流程图源代码

```
sequenceDiagram
    participant F as ■■■
    participant S as ■■
    participant Sv as surveys
    participant R as responses
    participant D as drafts
    F->>S: ■■■■■■■■■
    S->>S: ■■■■■■404 / ■■■ / ■■■ / ■■■■■■
    S->>S: replay ■■■■■■■■■■■
    S->>Sv: ■■■■■■■ responseCount + 1■■■■■■■■■■■
    alt ■■■■■■■■■■■
        Sv-->>S: ■■■
        S-->>F: 409 survey_ended
    else ■■■■■
        S->>R: ■■■■■
        alt ■■■■■■■■■■■■■
            S->>Sv: responseCount - 1
            S-->>F: 409 already_submitted
        else ■■
            S->>D: ■■■■■■■
            S-->>F: 201 ■■■■■■■■■
        end
    end
```

4.5 冗余的取舍

只冗余了 4 个字段，每个都写明了理由和保持一致的方法：

- `surveys.responseCount`：列表展示和上限控制需要一个能原子比较的值，可用"重新计数"校准；
- `responses.answers[].type`：统计时直接按题型处理，题型发布后不可改；
- `responses.singleSubmission`：部分索引只能根据本文档的字段过滤；
- `responses.surveyVersion`：区分新旧答卷。

题目文字、选项文字不冗余，统计以问卷当前内容为准。

4.6 断点续填

打开可填写的问卷时，按"一人一问卷"做 upsert。进度上的问卷版本和当前版本不一致就清空；一致则回到路径末尾那一题。每点一次"下一题"，用一条管道更新写入答案，并把路径截断到这道题再接上下一题。提交成功后删除进度；30 天没有再访问的进度由 TTL 索引删除。

流程图源代码

```
flowchart TD
    Open["■■■■■■■■■■■■■■■"] --> Touch["upsert■■■■■■■■■■■■■■■■■"]
    Touch --> Ver{"■■■■■■■■■■■■■■?"}
    Ver -->|■| Reset["■■■■■■■■■■■■■■■"]
    Ver -->|■| Resume["■■■■■■■■■■■■■■■■■■"]
    Resume --> Next["■■■■■■■■ → ■■■■■■■■■■■■■■■■■"]
    Reset --> Next
    Next --> Submit["■■■■■■■■■■■"]
    Next -.->|"30 ■■■■■"| TTL["TTL ■■■■■■■"]
```

5. 跳转设计

- 规则存储在问卷文档中：`{"when": {"op": "eq", "value": "<optionId>"}, "goto": "<questionId ■ END>"}`，另有"否则跳到" `defaultGoto`。程序中只有一个通用的规则解释器，不写死任何问卷的跳转。
- 运算符按题型提供：单选 3 种（选了 / 没选 / 选了其中之一）、多选 4 种（包含 / 包含任一 / 包含全部 / 恰好是）、数字 7 种（等于、不等于、大于、大于等于、小于、小于等于、介于）。

- 计算顺序：本题规则按顺序匹配，命中第一条即跳转；都不命中走 `defaultGoto`；再没有就按顺序进入下一道未隐藏的题；目标题已隐藏时顺延。
- 只能向后跳，路径长度不超过题目数，从结构上杜绝死循环；调整顺序导致向前跳时拒绝保存。
- 提交时服务端从第一题重放整条路径，拒绝路径之外的答案，防止绕过前端伪造路径。

详见关键逻辑说明第 1 节。

6. 校验设计

校验分三层，分别在不同时机拦截不同的问题。定义校验保证库里的问卷自洽，答案校验保证这一次作答合法，`$jsonSchema` 是绕过后端也写不进脏数据的底线。

流程图源代码

```
flowchart LR
    Edit["■■■■■■"] --> RC["rule_checker■■■■■■■■■■"]
    Fill["■■■■■ / ■■"] --> AV["answer_validator■■■■■■■■"]
    RC --> DB["MongoDB $jsonSchema■■■■■■■■■■"]
    AV --> DB
```

层次	时机	负责
问卷定义校验 `rule_checker`	编辑页每次保存，检查整份问卷	选项数、约束自洽、运算符适用于题型、比较值属于本题、跳转目标存在且在后面
答案校验 `answer_validator`	每点"下一题"和提交时	必答；单选选项有效；多选数量；文本字数（先去首尾空格）；数字范围、整数、拒绝 NaN 和超大整数
数据库 `$jsonSchema`	任何写入	必填字段、类型、枚举、格式，禁止未定义字段

错误提示精确到题目，例如"至少选择 2 项""必须为整数""最多输入 20 个字（当前 23 个）"。答案校验通过后返回规范化的值（字符串 "25" 变为整数 25），跳转判断和入库都用它。

详见关键逻辑说明第 2 节。

7. AI 使用过程

7.1 工具与工作方式

- 工具：第一阶段使用 Cursor（Agent/Plan 模式）；第二阶段使用 Codex 的规划、文件修改、PowerShell、pytest 和本地浏览器能力。两个阶段均把原提示词、结果和实际修改写入 AI 使用日志。
- 工作方式：

1. 先让 AI 制定计划；
2. 我在计划中加入"每完成一步就汇报、我确认后再继续"，避免 AI 一次性生成大量我无法审查的代码；
3. 每一步完成后 AI 跑全部测试、更新《AI 使用日志》、Git 提交，我审查后再确认下一步；
4. 第二阶段继续采用六步核验门槛，每步完成代码、测试和文档后，由我回复“下一步”才进入下一步。

- 全过程记录在《AI 使用日志》中；第一阶段 #1–#16 原记录保留，第二阶段从 #17 继续编号，每条包含提示词原文、工具、结果、是否修改和修改内容。

日志	内容
#1	规划整个作业
#2	初始化项目
#3	初始化之后我立即指出缺少界面分析，给出自己的页面流程图，并提出数据库要按页面来设计
#4、#5	围绕页面流程的两轮争论：我拒绝把页面并成六页，改了登录默认值、回跳、"你已填写过"和草稿外链，并定下一屏一题、只能向后跳
#6、#7	围绕数据库的两轮争论：我否掉拆题、答卷内嵌、用事务、发布后锁死等方案，确定内嵌题目、独立答卷、`settings`、`responseCount`、隐藏不删、部分唯一索引和 HMAC；AI 按此写入设计文档和 schema
#8 至 #10	认证、创作者后台、编辑器、跳转逻辑、填写提交、统计导出
#11、#12	前端页面与浏览器测试；我要求重新设计界面
#13、#14	我提出断点续填与"我参与的问卷"，AI 设计并实现
#15	整理测试用例
#16	第一阶段系统说明、API 说明、关键逻辑说明、README 和报告
#17–#20	第二阶段全工作区审计、需求规划，以及我要求统一第一、二阶段页面流程后的修订
#21–#24	数据迁移、题目版本与共享、多题库与问卷引用、跨卷统计及前端，每步单独核验
#25	最终回归、提交文档、AI 日志和报告 PDF

7.2 AI 帮了什么

- 规划与拆解：根据作业要求和评分权重拆出步骤；提醒第二阶段占 70%，计划里预留了 `schemaVersion` 这类扩展点。题目用稳定 ID、发布后只隐藏不删除，是我在 #3、#6 里定的。
- 数据库设计：在 #4 至 #7 的争论里提出过另外几套做法（题目独立成集合、答卷内嵌、用事务包住计数和插入、发布后禁止修改、答案里保存选项文字快照、"每人一次"先查再插）。最终结构按我的要求落实后，由 AI 写成 `$jsonSchema`、索引声明、设计文档和测试，并补上与关系型结构的对比。断点续填阶段，AI 设计了 `drafts` 集合、TTL 索引和管道更新截断路径。
- 代码实现：全部后端、前端、校验规则和测试代码由 AI 编写，约 5000 行。
- 测试：编写 242 个自动测试，包括并发测试和用 `explain` 验证索引使用的测试；用内置浏览器完整走通页面流程。
- 文档：生成需求分析、数据库设计、API 说明、关键逻辑说明、测试用例等文档初稿。

7.3 AI 做错了什么

问题	怎么发现的	怎么处理的
计划里没有界面和访问模式分析，初始化后仍准备直接设计三个集合	我看完初始化结果后立即指出	停掉写 schema，先用我画的页面流程图定访问模式（日志 #3）
把页面收成 6 个：列表直达编辑，分享设置并进编辑页，取消预览	对照我的流程图	管理页保留为中心，预览、分享、统计、导出分开（日志 #4）
建议登录后直接进入填写页；跳转可以跳到任意题，保存时再查环	讨论流程时	登录后回到链接入口重新检查状态；只能向后跳（日志 #4、#5）
建议题目独立成集合、答卷内嵌或用事务、不存 `responseCount`、发布后禁止修改、答案里冗余选项文字	讨论数据库时	题目内嵌、答卷独立、条件更新占名额、发布后隐藏不删、统计以当前问卷文字为准（日志 #6）
建议"每人一次"在代码里先查再插，匿名问卷无法去重	讨论索引时	唯一部分索引；匿名用 HMAC，能去重但不能反推身份（日志 #7）
导出排序只按提交时间，同一毫秒的答卷顺序不稳定	自动测试失败	加 `_id` 作为第二排序键并调整索引
初始化脚本无法重建定义变化的同名索引	修改索引后报错	比较索引定义，不同则删除重建
时间带微秒，与 MongoDB 的毫秒精度不一致，乐观锁永远冲突	编写乐观锁时发现	生成时间时截断到毫秒
同一毫秒内两次保存的 `updatedAt` 相同，乐观锁失效	全量测试偶发失败	管道更新使 `updatedAt` 严格递增
查询"是否已填过"时没有带上部分索引的过滤条件，用不上索引	核对执行计划	补上条件并用测试验证
并发测试看似覆盖了"占名额失败"，实际上那段代码从未执行	覆盖率报告	补充结果确定的测试
设计文档声称"只在 repositories 层访问数据库"，但有两处服务直接查库	编写系统说明时核对	把两处查询移到 repositories
错误提示带英文前缀；错误提示在页面底部看不到；窄屏布局错乱	浏览器测试	逐项修正
界面过于简陋，是通用的白卡片加蓝按钮	我提出	以"答题卡"为题材重新设计
演示数据脚本第一次运行报错；修改测试文件时误加了重复测试；把覆盖率临时文件提交进仓库	运行时报错 / 自查	当即修正

问题	怎么发现的	怎么处理的
测试中同时发出两个有先后依赖的浏览器操作，顺序颠倒，误以为程序出错	查看服务端日志	改为逐步操作

7.4 我改了什么、决定了什么

- 工作流程：要求逐步汇报、确认后再继续；在 AI 每步汇报后审查结果。初始化之后没有同意直接设计集合。
- 设计方法：初始化一结束就指出缺少界面分析，要求先确定页面和每个页面的信息、操作，再设计数据库。
- 页面流程：自己绘制了页面流程图，包括问卷管理页作为中心、预览、分享设置、导出、外链结束页，以及 404 / 已结束等状态分支。AI 要把页面并成六页，我没有接受。讨论之后我自己改了流程图：填写默认要求登录但保留开关、登录后回到链接入口、增加"你已填写过"、草稿外链改为 404，并定下一屏一题、只能向后跳。
- 数据库：自己提出并在争论里维持的结构是：题目内嵌在问卷中，答卷单独成集合，分享设置收成 `settings` 子文档，用 `responseCount` 做上限而不是现算或事务，发布后删除改为隐藏、题型不可改、结构变更 `version + 1`，问卷软删除。索引上要求唯一部分索引实现"每人一次"，匿名用 HMAC，列表索引带上 `deletedAt`，编辑用 `updatedAt` 做乐观锁。AI
  提出的拆表、答卷内嵌、多文档事务、发布后锁死、选项文字快照、先查再插，都没有采用。
- 界面：认为初版界面过于简陋，要求重新设计。
- 新功能：提出"缓存最近访问的问卷和答题记录、在个人主页继续填写或查看"的需求，确认 AI 的方案后实现。

7.5 体会

这次迭代让我更清楚地看到，AI
最擅长快速展开实现和把大量边界变成测试，但它不能代替需求口径的决定。第二阶段一开始，AI 单独画了题库局部流程；我对照第一阶段页面后追问新功能如何进入原管理页、编辑页和统计页，才把两套流程统一。这个过程说明“先页面和访问模式，后数据结构”仍然有效：如果入口、返回位置、权限和错误状态没有先确定，集合拆分看起来合理，用户实际使用时仍会断裂。

逐步核验也很重要。每一步限定范围，使数据库迁移、版本共享、题库引用、统计和前端可以分别检查，问题不会堆到最后。AI 在第 5 步实现了撤销共享后的统计读取，但详情页权限一度没有同步；浏览器和针对性测试把这个不一致暴露出来。最终回归时，一次被中断的测试进程还造成两个 pytest 同时清理测试库，错误表面上像大量功能失效；查看异常的共同模式后才确认是环境竞争，并在单进程重跑后得到可信结果。这也提醒我，测试数字必须连同环境和过程一起解释，不能只看最后一行。

AI 生成的代码仍需要我判断哪些设计符合课程要求。例如旧答卷没有当时的题目快照，无法可靠推回题目版本；最终没有为了让统计数字“更完整”而伪造归版，而是保留原单卷统计并在跨卷统计中单列。这类取舍比单纯多做功能更能体现数据库演进中的责任边界。

8. 测试结果

详见测试用例。第一阶段输出保存在 `docs/test-results/pytest-output.txt`，第二阶段最终输出保存在 `docs/test-results/pytest-output-stage2.txt`。

- 第一阶段基线：242 个全部通过，用时 73.55 秒；后端代码覆盖率 98%。
- 第二阶段最终结果：268 个全部通过，用时 169.07 秒；1954 条语句中未覆盖 80 条，覆盖率 96%。

- 覆盖范围：作业要求的六类，以及认证与权限、数据库约束与索引、并发、断点续填、幂等迁移、版本分叉/恢复、共享撤销、多题库、引用隐私、新答卷归版和同版跨卷统计。
- 几项关键结果：
- 8 个用户同时争抢 3 个名额，恰好 3 个成功；同一用户 6 个线程同时提交，恰好 1 个成功，计数器正确；
- 5 份固定答卷的统计结果与手算一致（如平均年龄 27.5、苹果 100%）；
- 伪造作答路径被拒绝；
- 列表、统计、导出、"我参与的"等查询都使用索引且没有内存排序。
- 手工测试：第一阶段完成 13 项浏览器测试；第二阶段在临时数据库完成命名题库、创建题目、建立 v2、将 v1 选入新问卷、发布冻结、复制为草稿和版本统计展示，测试后清理临时数据库。

页面	截图
编辑页：题目与跳转规则	编辑页（见截图附录）
统计页	统计页（见截图附录）
我参与的问卷	我参与的问卷（见截图附录）

9. 遇到的问题及解决方法

问题	解决
设计数据库时缺少依据	先做页面与访问模式分析（P1 至 P23），再按访问模式决定结构和索引
单机 MongoDB 没有多文档事务，提交答卷涉及计数器和答卷两个集合	"条件原子更新占名额 → 插入答卷 → 失败归还"，只依赖单文档原子性；提供"重新计数"修正极端情况
多人同时提交可能超过上限、同一人可能重复提交	条件原子更新；唯一部分索引
匿名与"每人一次"矛盾：不存身份就无法去重	`submitterKey` 用 HMAC 生成：可去重、不可反推
发布后修改问卷会使旧答卷对不上	题目和选项用稳定 ID；删除改为隐藏；不可改题型；版本号记入答卷
跳转可能形成死循环	只允许向后跳，保存时检查
同一毫秒内的数据导致排序不稳定、乐观锁失效	加 `_id` 作为第二排序键；`updatedAt` 严格递增
部分索引用不上	查询中带上与过滤条件一致的谓词
修改索引定义后无法自动更新	`init_db` 比较定义后重建
统计要同时得出多种结果	`$facet` 一次聚合；单题统计复用同一管道
以 $ 开头的文本在管道更新中被当成字段引用	写入值用 `$literal` 包裹
测试太慢	测试中降低 bcrypt 轮数

10. 【第二阶段新增】需求变化与系统演进

10.1 第一阶段设计和新需求造成的问题

第一阶段为“打开并编辑一份问卷”优化：题目、选项、约束和跳转规则全部内嵌在 `surveys.questions`；答卷独立存于 `responses`，只用本卷 `questionId/optionId` 指向当前问卷内容。这个结构让填写时一次读取即可得到整卷，也能依赖单文档原子更新完成编辑。

第二阶段要求同一题跨问卷复用、分享、查看版本历史和引用，并按同一题的同一版本跨卷统计。原结构没有独立的“题目身份”，两个文字相同的题无法判断是否是同一道题；直接让问卷引用一个可变题目又会导致旧卷随新修改变化。旧答卷也没有提交时的题目文字快照，不能可靠知道它对应迁移后的哪个版本。因此必须增加身份和版本层，同时继续保留问卷快照。

10.2 修改理由和核心方案

最终采用“独立版本 + 问卷快照”的混合结构：

- `question_items` 表示稳定题目身份、所有者和共享权限；
- `question_versions` 保存不可变正文、版本号及父版本；可从任意旧版分叉，恢复也是创建新版本；
- `question_banks` 只负责用户的多个命名分类，一个题目可进入多个题库；
- 问卷仍保存完整题目快照、顺序、隐藏状态和跳转规则，并增加来源题目/版本 ID；
- 新答卷为实际回答写具体 `questionVersionId`，支持精确跨卷统计。

这样，填写和原跳转逻辑继续只依赖问卷快照；题库和版本管理不会在运行时改变旧问卷。已发布或已关闭问卷的题目、选项、顺序和规则冻结，需要修改时复制为新草稿。

10.3 数据结构、索引与迁移

集合由 4 个扩展为 7 个，`schemaVersion` 升为 2。保留第一阶段 9 个索引，新增题目归属、公开/定向共享、来源唯一键、版本唯一键、题库名称唯一键、引用查询和版本答案查询索引，共 17 个业务索引（不计 `_id`）。七个集合均有 `$jsonSchema`。

迁移脚本先只读预检，再生成 BSON ZIP 备份并校验 SHA-256，最后按 `(surveyId, questionId)` 为每个现存题建立独立身份和第 1 版。正式库迁移前后保持 8 个用户、4 份问卷、10 份答卷、1 份草稿；11 道现存题生成 11 个身份和 11 个初始版本。重复执行没有新增或重复记录。旧用户、答卷和草稿 BSON 内容不变，原问卷只增加来源字段。

旧答卷缺少当时题目快照，因此不事后补写推测的版本 ID。它们继续参与原问卷统计；在跨卷统计中只计入“无法归版”数量，不进入选项分布或数值平均值。

10.4 程序与页面调整

后端增加题目身份/版本、共享、题库、引用和跨卷统计的模型、仓储、服务与路由。发布前自动为尚未关联的草稿题建立私有来源；从题库选题时指定具体版本并复制为问卷快照；共享撤销后禁止新引用，但已有问卷继续按快照使用。引用查询对本人问卷显示明细，对他人问卷只显示去重数量；跨卷统计只处理当前用户自己的未删除问卷。

前端没有另做一套系统，而是在第一阶段流程中接入新能力：主导航增加“我的题库”；编辑页增加内嵌选题器和“保存到题库”；问卷管理页增加题目来源与“复制为新草稿”；原单卷统计页提供来源版本入口。新增 `/question-banks`、`/question-banks/{id}` 和 `/questions/{id}` 三类页面，题目详情集中处理版本、共享、引用和跨卷统计。填写页、分享页、个人答卷与原统计路径继续沿用。

10.5 兼容性问题和处理

兼容问题	处理方式
第一阶段允许发布后改题，第二阶段要求冻结	更新编辑 API 为明确的 409；标题、说明和填写设置仍按原接口管理；需要改题时复制问卷
旧问卷没有来源 ID	迁移时按每个现存问卷题分别建立身份，不按标题自动合并
旧答卷没有题目版本	原单卷统计照常读取；跨卷统计单列，绝不推测归版
撤销共享后的旧引用	新选题和复制被拒；已有本人问卷可查看来源历史和自己的统计
共享题的他人使用情况可能泄露	只返回他人问卷数量，不返回标题、答卷或所有者
版本内容与本卷跳转规则边界	版本只保存题目正文；顺序、隐藏和规则继续归问卷快照

10.6 如何保证旧功能仍可用

原路由和填写、跳转、校验、提交、统计、导出、断点续填、参与记录仍保留。受新需求明确改变的只有发布后题目编辑行为，相应旧测试改为断言冻结错误，并单独说明原因。最终全量测试收集 268 项，全部通过，覆盖率 96%；其中包含第一阶段所有未被新需求改变的回归测试，以及第二阶段新增的迁移、版本、共享、题库、引用和统计测试。

10.7 第二阶段实施过程

实施分为六步，每步结束都提交文件清单、关键差异、测试结果和待核验点，收到我的“下一步”后才继续。第 1 步固定基线并统一页面流程；第 2 步完成 schema、备份和幂等迁移；第 3 步完成版本与共享；第 4 步完成多题库、引用、复制和冻结；第 5 步完成答卷归版、跨卷统计和前端；第 6 步做最终回归、文档统一和 PDF。逐文件原状、改动、原因和需求映射见《第二阶段变更清单》。

10.8 已知限制

- 迁移前已经发生过的题目文字变化无法从现有答卷还原；报告和统计均明确承认这个历史边界。
- 题目版本使用不可变插入，版本号表示创建顺序，真正的分叉关系由 `parentVersionId` 表示。
- 跨卷统计只合并当前用户自己的问卷，不提供团队级答卷共享，避免共享题目泄露他人数据。
- 前端使用原生 JavaScript，并在 Chromium 内核浏览器核验；没有覆盖所有浏览器。

10.9 最终提交材料对应关系

作业要求	本项目文件
可运行代码、MongoDB schema、配置说明	`backend/`、`frontend/`、`schema/`、`README.md`
系统说明、数据库设计、API、关键逻辑	`docs/■■■■.md`、`■■■■■.md`、`API■■.md`、`■■■■■■.md`
测试步骤、输入、输出、结果	`docs/■■■■.md` 与两个 pytest 输出文件
完整报告	本报告 Markdown 与第二阶段 PDF
真实 AI 使用过程	`docs/AI■■■■.md`；原提示词、工具、结果和修改按时间记录
第二阶段修改标注	`docs/■■■■■■■■.md`，逐文件标记【第二阶段新增/修改】

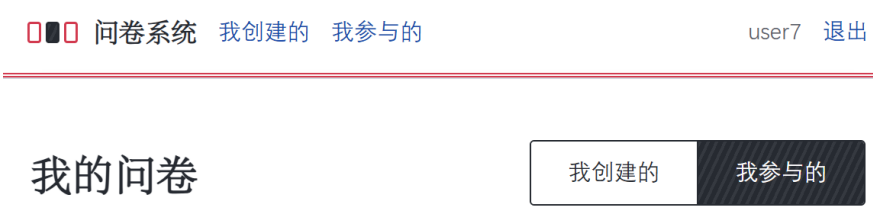
附录：代表性界面截图



第一阶段：问卷编辑与跳转规则



第一阶段：单问卷统计



第一阶段扩展：我参与的问卷